Colab × Gemini

中文 NLP 實作教學講義

用中文指揮 AI 寫程式｜先思考、後驗證｜CRISP 提示框架

項目	內容
對象	大一資管、非資工科系學生（不需程式基礎）
時數	8 單元 × 100 分鐘，外加期末專題展示
工具	Google Colab（雲端 Python 筆記本）＋ Gemini（對話面板與程式呼叫）
教學法	先思考、後驗證；人在迴路（HITL）監督；提示詞採 CRISP 五面向
期末產出	一份可執行的中文文本分析或文件問答筆記本，附提示日誌與監督日誌
版本	2026 年 11 月（Colab 與 Gemini 功能依當月官方文件查核）
編撰	郭博仁　國立屏東科技大學資訊管理系

內容一覽：1 工具說明　2 課前準備　3 CRISP 與開發迴圈　4 單元 1–8　5 評量　6 影音資源　7 常見問題　8 參考資料

1　課程定位與工具說明

本講義把 Google Colab 當成「用中文下需求、由 Gemini 產生程式、學生執行並驗證」的開發平台，並以中文自然語言處理（NLP）為實作題材。學生的角色從「寫程式的人」轉為「提出需求、審核計畫、驗證結果的監督者」。

「自然語言」在本課有兩層意思：**開發介面是自然語言**（用中文對 Gemini 描述需求）；**處理對象也是自然語言**（斷詞、詞頻、情感、摘要、文件問答）。



圖 1　在 Colab 使用 Gemini 的兩種方式

> **使用前必讀**
>
> Colab 的 AI 功能需要 Google 帳號**年滿 18 歲**；未滿 18 歲的學生由教師示範，或改用教師代管的 Gemini 服務。
>
> 官方說明：提示、程式碼與輸出會被收集用於改善產品，並可能經人工審閱、保存最長 18 個月。**不要輸入個資、成績與未公開資料。**
>
> 免費帳號的 Colab 執行時間、閒置中斷與 `google.colab.ai` 用量上限會變動，官方不公布固定數字。

1.1　與 Claude Code 版講義的對照（教師參考）

功能	Claude Code 版	Colab＋Gemini 版
下達需求	在終端機與 Claude 對話	Gemini 對話面板
先提計畫	計畫模式（Plan mode）	提示寫「先列計畫，不要寫程式」；Data Science Agent 先提計畫再執行

功能	Claude Code 版	Colab＋Gemini 版
專案規則	CLAUDE.md	Custom Instructions＋筆記本開頭的文字儲存格
可重用流程	技能（Skills）	函式＋範本筆記本
執行程式	AI 執行，學生核准	學生自己按 ▶ 執行
回復舊版	/rewind	檔案 → 修訂版本記錄
呼叫 LLM 處理文字	claude -p／Claude API	google.colab.ai（免金鑰）／Gemini API

2　課前準備（詳細步驟）

第一次上課前完成 7 個步驟，約 30 分鐘。建議先看影音資源 V3、V8 或 V9。

步驟 1　開啟 Colab 並建立筆記本

1. 用 Google 帳號登入，瀏覽器開啟 `colab.research.google.com`。
2. 選「新增筆記本」（或 檔案 → 新增筆記本）。
3. 點左上角檔名，改為 `nlp_lab.ipynb`。筆記本會自動存在雲端硬碟的「Colab Notebooks」資料夾。

步驟 2　認識介面



圖 2　Colab 介面導覽（示意圖）

步驟 3　連線並執行第一個儲存格

1. 點右上角「連線」，等待出現已連線與 RAM／磁碟指示。
2. 在程式碼儲存格輸入下方程式，按 ▶ 或 Shift+Enter 執行。
3. 看到輸出就代表環境正常。

> **Colab 程式碼儲存格**
>
> ```
> print("Hello, Colab! ")
> ```

步驟 4　準備資料：上傳或掛載雲端硬碟

左側 📁 可直接上傳 CSV，但**上傳到 /content 的檔案在執行階段結束後會消失**。整學期使用的資料，建議放在雲端硬碟的 `nlp-lab` 資料夾，再掛載：

Colab 程式碼儲存格

```
# 掛載 Google 雲端硬碟（資料放在「我的雲端硬碟/nlp-lab」）
from google.colab import drive
drive.mount("/content/drive")
%cd /content/drive/MyDrive/nlp-lab
```

步驟 5　設定 Gemini 對話面板

1. 點頁尾的 Gemini 星形圖示，開啟右側對話面板。
2. 在對話框開啟 Custom Instructions，貼上下方課程規則並存到筆記本；之後在這本筆記本的對話都會套用。
3. 想練習自己寫程式時，開啟 Learn Mode：Gemini 只給步驟提示與概念說明，不直接給完整程式。
4. 程式出錯時，可請 Gemini 解釋錯誤；修正會以差異對照呈現，**看懂再接受**。

Custom Instructions 範本

請用繁體中文回答。
我是大一學生，正在學中文 NLP（jieba 斷詞、pandas）。
收到需求時，先列出計畫與要安裝的套件，等我回覆「可以」再寫程式。
程式加上中文註解；只用我提供的資料，不要自己編數字。
不要使用舊套件 google.generativeai；呼叫 Gemini 請用 google.colab.ai。

步驟 6　（選用）設定自己的 Gemini API 金鑰

一般課堂用 `google.colab.ai` 即可（免金鑰）。需要指定較新模型或較大用量時，才改用 API 金鑰。



圖 3　Gemini API 金鑰設定流程

Colab 程式碼儲存格（進階 B2）

```
# 進階：以自己的 Gemini API 金鑰呼叫（金鑰存在 Colab Secrets）
!pip install -q -U google-genai
from google import genai
from google.colab import userdata

client = genai.Client(api_key=userdata.get("GEMINI_API_KEY"))
r = client.interactions.create(model="gemini-3.8-flash",
                               input="用一句話說明什麼是中文斷詞")
print(r.output_text)
```

上例模型為 Gemini API 文件 2026 年 10 月的範例 `gemini-3.8-flash`；模型名稱會更新，以官方模型頁為準。

步驟 7　課堂安全守則

1. AI 產生的每段程式，執行前先讀過；看不懂的那行先請 Gemini 解釋。
2. 資料須去識別化；不輸入個資、成績、帳號密碼與 API 金鑰。
3. 分享筆記本前：編輯 → 清除所有輸出，避免洩漏資料。
4. 不用爬蟲抓取他人評論；使用課程提供或開放資料。
5. AI 的結論一律驗證後才寫進報告，並記入監督日誌。

3　CRISP 提示框架與開發迴圈

全課提示一律依 CRISP 五面向撰寫。**P（驗證）一定要變成可執行的測試**，不接受 AI 口頭說「已驗證」。



圖 4　CRISP 提示五面向

CRISP 提示模板

C 背景：我是____系__年級學生，正在做____；領域細節：____。
R 角色：請扮演____，讀者是____，語氣____。
I 任務：請____。輸入：____　處理：____　輸出：____
S 規格：輸出格式____；範例一列：____；條件：____（只用我給的資料）。
P 驗證：測試資料____；預期結果由我填寫____；
　　檢查等號、空值、負數、單位；不可修改預期值。

每個單元都走同一個迴圈。與 Claude Code 版不同的是：Colab 的程式由學生自己按 ▶ 執行，所以「執行前讀懂程式」也是學生的責任。

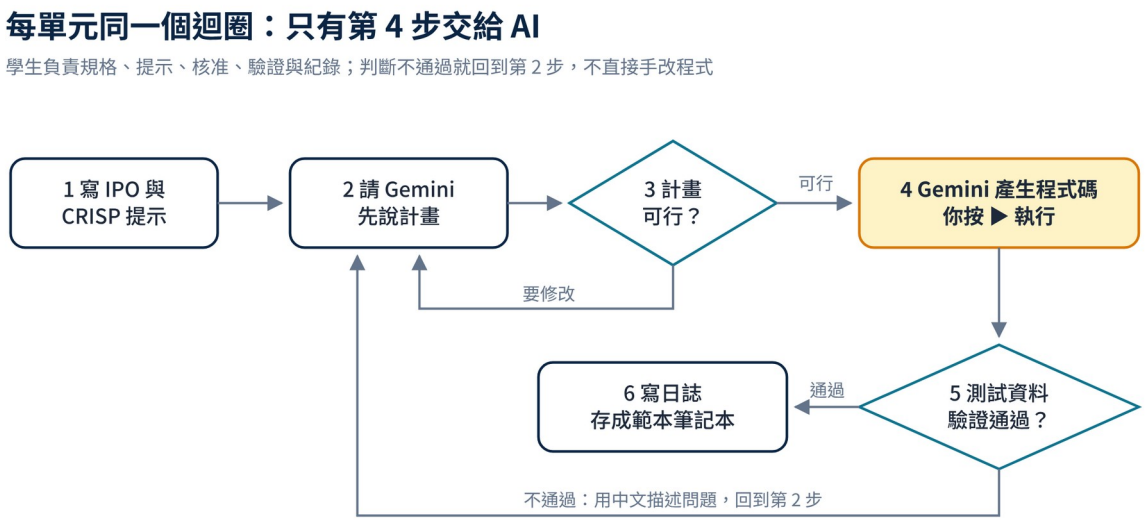


圖 5　每單元開發迴圈



圖 6　中文 NLP 處理管線與單元對應

單元	主題	Colab／Gemini 重點	NLP 實作	產出
1	從寫程式到說需求	Gemini 面板產碼、執行儲存格	字數、標點、句數	第一本筆記本
2	先思考再交辦	IPO、先列計畫再產碼	jieba 斷詞、詞頻	freq.csv
3	驗證就是學習	人寫預期值、修訂版本記錄	自訂詞典、停用詞、魚塭警示測試	測試結果
4	讓資料說話	描述現象請 Gemini 修錯	長條圖、文字雲	top15.png
5	情感分析	google.colab.ai 呼叫 Gemini	詞典法 vs. Gemini	準確率表
6	把好流程存起來	Custom Instructions、函式、範本	一行完成分析	範本筆記本
7	LLM 當 NLP 引擎	批次呼叫、JSON 解析	摘要、分類、幻覺檢查	結果表
8	整合專題	檢索＋生成	文件問答	期末筆記本

4　單元講義

每單元包含：目標、操作步驟、Gemini 提示、參考程式碼、執行結果、HITL 檢核點與預習影音。參考程式碼已用範例資料實測可執行。

單元 1｜從寫程式到說需求：第一本 Colab 筆記本

目標：用中文請 Gemini 寫出字數統計程式，學生自己執行，並與手算結果比對。

操作步驟

1. 檔案 → 新增筆記本，命名為 `U1_字數統計.ipynb`。
2. 新增文字儲存格寫 IPO：輸入一段中文；處理：計算字數、標點、句數；輸出：四個數字。
3. 先手算範例段落的句數與標點數，寫在文字儲存格。
4. 開啟 Gemini 面板，貼上下方提示；確認做法後回覆「可以」。
5. 將 Gemini 產生的程式插入新的程式碼儲存格，讀過一遍再按 ▶ 執行。
6. 與手算比對；不一致時用中文描述差異請 Gemini 修正，不自己改程式。

> **Gemini 提示（CRISP）**
>
> C：我是大一學生，第一次用 Colab 寫 Python。
> I：計算一段中文的 1 總字數（不含空白）2 中文字數 3 標點數（全形半形都算）4 句數（以。！？計）。
> S：段落放在變數 text；結果印成一行。
> P：先用三行說明做法，等我回覆「可以」再寫程式。

Colab 程式碼儲存格

```
# 單元 1：中文字數統計
import re
text = """屏東科技大學位於內埔。校園很大，騎腳踏車要十分鐘！你去過嗎？"""
total = len(re.sub(r"\s", "", text))                    # 去除空白與換行
chinese = len(re.findall(r"[\u4e00-\u9fff]", text))  # 中文字
punct = len(re.findall(r"[，。！？、；：「」（）,.!?;:]", text))
sentences = len(re.findall(r"[。！？!?]", text))
print(f"總字數 {total}，中文字 {chinese}，標點 {punct}，句數 {sentences}")
```

執行結果

```
總字數 30，中文字 26，標點 4，句數 3
```

HITL 檢核點

- 程式算的句數與手算一致嗎？不一致時是誰的定義不同？
- 半形「?」與全形「？」都有算到嗎？
- 你能說出每一行在做什麼嗎？看不懂的請 Gemini 逐行解釋。

預習影音：V3、V10

單元 2｜先思考再交辦：斷詞與詞頻

目標：先寫 IPO，請 Gemini 只列計畫；學生修改並核准後才產生程式，得到詞頻前 20 名。

中文句子沒有空格，NLP 第一步是斷詞。`jieba` 預設詞典偏簡體，繁體文本改用大詞典 `dict.txt.big`。

操作步驟

1. 左側 📁 上傳 `feedback.csv`（欄位 id、建議），或使用掛載的雲端硬碟資料夾。
2. 執行安裝儲存格，下載 jieba 與繁體大詞典。
3. 在 Gemini 面板貼上提示，要求「只列計畫」。
4. 逐條檢查計畫，至少修改一處（例如：排除單字詞、只讀「建議」欄）。
5. 回覆「可以」後產生程式碼，讀過再執行。
6. 檢查前 20 名，圈出沒有分析意義的詞，留到單元 3 處理。

Colab 程式碼儲存格（安裝）

```
!pip install -q jieba
!wget -q -O dict.txt.big
https://github.com/fxsjy/jieba/raw/master/extra_dict/dict.txt.big
```

Gemini 提示（CRISP）

C：feedback.csv 是課程回饋，欄位 id、建議，UTF-8 編碼。
I：輸入「建議」欄 → 用 jieba（繁體大詞典 dict.txt.big）斷詞，去除單字詞、標點與數字 → 計算詞頻。
S：輸出 freq.csv（欄位：詞, 次數，依次數遞減），並顯示前 20 名。
P：先只列出執行計畫與需要安裝的套件，說明每個套件的用途，不要寫程式。

Colab 程式碼儲存格

```
# 單元 2：斷詞與詞頻
import jieba, pandas as pd
from collections import Counter
jieba.set_dictionary("dict.txt.big")      # 繁體大詞典

df = pd.read_csv("feedback.csv")           # 欄位：id, 建議
words = []
for t in df["建議"].dropna():
    for w in jieba.lcut(t):
        if len(w) >= 2 and not w.isdigit():  # 去除單字詞、標點與數字
            words.append(w)

freq = Counter(words).most_common(20)
pd.DataFrame(freq, columns=["詞", "次數"]).to_csv(
    "freq.csv", index=False, encoding="utf-8-sig")
freq
```

執行結果（範例資料）

```
[('希望', 5), ('多一些', 3), ('練習', 3), ('可以', 2), ('範例', 2),
 ('實作', 2), ('Python', 2), ('清楚', 2), ('作業', 2), ('說明', 2), …]
```

HITL 檢核點

- 計畫有沒有讀錯欄位或要裝不需要的套件？
- 「希望」「可以」對分析有意義嗎？該不該進停用詞？
- 範例中「老師上課」被切成一個詞，合理嗎？

預習影音：V4、V5

單元 3｜驗證就是學習：測試案例與魚塭案例

目標：用「人寫的預期值」驗證 AI 程式，並辨識 AI 自稱驗證通過的陷阱。程式改壞時，用 檔案 → 修訂版本記錄 回到先前版本。

A. 斷詞測試：自訂詞典與停用詞

1. 各組寫 5 句本系領域句子，並在紙上寫出正確斷詞。
2. 請 Gemini 建立自訂詞典與停用詞檔，並寫測試；**正確答案由你提供，不可讓 Gemini 改**。
3. 執行測試，全部通過後，比較加詞典前後的詞頻前 20 名。

Gemini 提示（CRISP）

這句被切錯了：「屏東科技大學的水產養殖系研究石斑魚虹彩病毒」
正確應為：屏東科技大學／水產養殖系／研究／石斑魚／虹彩病毒（「的」列為停用詞）。
請建立 userdict.txt 與 stopwords.txt，寫一個 seg() 函式，
並用我給的正確答案寫測試、印出通過或失敗。不可修改我的正確答案。

Colab 程式碼儲存格

```
# 單元 3：自訂詞典＋停用詞＋測試案例
with open("userdict.txt", "w", encoding="utf-8") as f:
    f.write("屏東科技大學\n水產養殖系\n石斑魚\n虹彩病毒\n")
with open("stopwords.txt", "w", encoding="utf-8") as f:
    f.write("的\n了\n很\n老師\n希望\n")
jieba.load_userdict("userdict.txt")
stop = set(open("stopwords.txt", encoding="utf-8").read().split())

def seg(t):
    return [w for w in jieba.lcut(t) if w.strip() and w not in stop]

cases = {   # 正確答案由「人」填寫，不可讓 AI 改
    "屏東科技大學的水產養殖系研究石斑魚虹彩病毒":
        ["屏東科技大學", "水產養殖系", "研究", "石斑魚", "虹彩病毒"],
}
for s, ans in cases.items():
    got = seg(s)
    print("通過" if got == ans else "失敗", got)
```

執行結果（範例資料）

```
通過 ['屏東科技大學', '水產養殖系', '研究', '石斑魚', '虹彩病毒']
```

B. 教學案例：AI 說「驗證通過」，其實沒有

教材為 CRISP「魚塭水質警示」任務卡交給 Gemini 的實際回答。它自稱 4 組測試全部一致，但對照提示逐項檢查，至少違反 7 項要求。

#	提示要求（面向）	Gemini 實際回答	問題類型
1	不要加表格外的說明（S）	表格前有大段優缺點比較與總結	忽略格式
2	只用給的資料，不自編數字（S）	自行加上 06:00–08:00 時間，並出現測試資料沒有的 30℃	自編資料
3	在 3 mg/L 畫紅色警戒線（S）	沒有圖，也沒說明做不到	條件遺漏
4	溶氧「低於」3 開增氧機（I）	3.0 mg/L 自行新增「接近警戒線」等級	擅改規格

#	提示要求（面向）	Gemini 實際回答	問題類型
5	列出預期結果並比對（P）	預期結果由 AI 自己填並套用自改規則，「一致」全為「是」	循環驗證
6	溶氧 4 值 × 水溫 2 值（P）	兩兩組合應有 8 組，只測 4 組，漏掉「2.4＋33」雙警示	測試不完整
7	檢查等號、空值、負數、單位（P）	只談溶氧等於 3；水溫恰為 32、空值、負數皆未測	自我檢查不完整

提示本身也要改：C 寫「我是學生」，R 又寫「對象是農友」，Gemini 因此開頭就稱使用者為農友。背景與讀者要分開寫清楚。

依原規則的正確預期（學生先自己填，再對答案）

溶氧＼水溫	28℃	33℃
5.2 mg/L	正常	水溫過高提醒
4.0 mg/L	正常	水溫過高提醒
3.0 mg/L	正常（規則是「低於」3）	水溫過高提醒
2.4 mg/L	開增氧機	開增氧機＋水溫過高提醒

3.0 該不該警示，是規則設計者（人）的決定；AI 可以提出疑問，不能自行改規則。

Colab 版：把驗證變成可執行的測試

> **Gemini 提示（CRISP）**
>
> C：吳郭魚魚塭監測專題，感測器回傳溶氧（mg/L）與水溫（℃）。
> R：警示訊息給農友看，用白話短句。
> I：寫函式 alert(do, temp)：溶氧 < 3 → 「開增氧機」；水溫 > 32 → 「水溫過高提醒」；
> 兩者可同時成立，用「＋」連接；都不成立 → 「正常」。
> S：空值、負數或非數字回傳「資料異常」，不可猜測。
> P：我會提供 expected 清單（含 8 組組合與邊界），請寫迴圈逐筆比對並印出通過或失敗。
> 不可修改 expected；規則不清楚先問我，不可自行新增警示等級。

Colab 程式碼儲存格

```
# 魚塭水質警示：函式＋人工預期值測試
def alert(do, temp):
    ok = all(isinstance(x, (int, float)) for x in (do, temp))
    if not ok or do < 0 or temp < 0:
        return "資料異常"
    msgs = []
    if do < 3:
        msgs.append("開增氧機")
    if temp > 32:
        msgs.append("水溫過高提醒")
    return "＋".join(msgs) if msgs else "正常"

expected = [   # (溶氧, 水溫, 預期結果)：學生自己填
    (5.2, 28, "正常"), (5.2, 33, "水溫過高提醒"),
    (4.0, 28, "正常"), (4.0, 33, "水溫過高提醒"),
    (3.0, 28, "正常"), (3.0, 33, "水溫過高提醒"),
    (2.4, 28, "開增氧機"), (2.4, 33, "開增氧機＋水溫過高提醒"),
    (3.0, 32, "正常"), (None, 30, "資料異常"), (-1, 30, "資料異常"),
]
for do, t, exp in expected:
    got = alert(do, t)
    print("通過" if got == exp else "失敗", do, t, "預期", exp, "實際", got)
```

執行結果（範例資料）

```
通過 5.2 28 預期 正常 實際 正常
通過 2.4 33 預期 開增氧機＋水溫過高提醒 實際 開增氧機＋水溫過高提醒
通過 3.0 32 預期 正常 實際 正常
通過 None 30 預期 資料異常 實際 資料異常
…（共 11 筆全部通過）
```

課堂活動（15 分）

發下 Gemini 原回答，各組 10 分鐘依 CRISP 五面向找錯，再對照上表。

最後執行 Colab 版測試，比較「口頭宣稱驗證」與「執行測試驗證」的差別。

HITL 檢核點

- 預期值是誰填的？測試通過，是程式對了，還是預期值被改了？
- 自訂詞典會不會讓其他句子切錯？多找兩句驗證。
- 回到修訂版本後，程式真的回到原樣了嗎？

預習影音：V6

單元 4｜讓資料說話：中文圖表與文字雲

目標：畫出詞頻長條圖與文字雲；練習「描述現象」請 Gemini 找原因，而不是貼一大段程式。

1. 先不裝字型直接畫圖，觀察中文變成方框（豆腐字）。
2. 在 Gemini 面板只描述現象：「圖上的中文都變成方框」，請它說明原因並修正。
3. 執行字型安裝儲存格（約 1 分鐘），再執行繪圖儲存格。
4. 把圖交給同組同學，只看圖說出三個發現。

Colab 程式碼儲存格（安裝中文字型）

```
!apt-get -qq update && apt-get -qq install -y fonts-noto-cjk
!pip install -q wordcloud
```

Colab 程式碼儲存格

```
# 單元 4：中文長條圖與文字雲
import matplotlib.pyplot as plt
from matplotlib import font_manager
from wordcloud import WordCloud

FONT = "/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc"
font_manager.fontManager.addfont(FONT)
plt.rcParams["font.family"] = font_manager.FontProperties(fname=FONT).get_name()

top = pd.read_csv("freq.csv").head(15)
plt.figure(figsize=(8, 5))
plt.barh(top["詞"][::-1], top["次數"][::-1], color="#0E7490")
plt.title("課程回饋高頻詞（前 15 名）")
plt.xlabel("次數")
plt.tight_layout(); plt.savefig("top15.png", dpi=150); plt.show()

wc = WordCloud(font_path=FONT, background_color="white", width=800, height=400)
wc.generate_from_frequencies(dict(zip(top["詞"], top["次數"])))
wc.to_file("cloud.png")
```

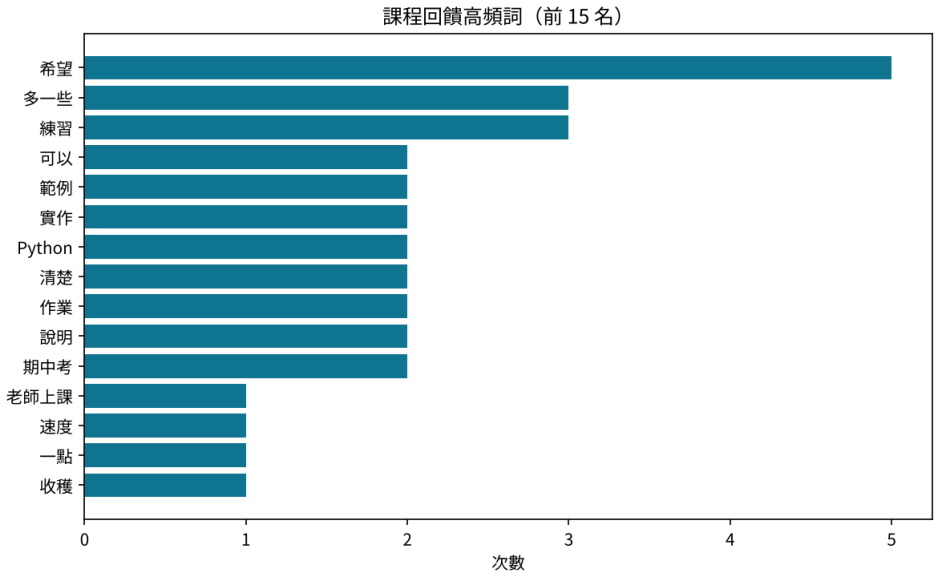


圖 7　單元 4 執行結果：課程回饋高頻詞（範例資料）

HITL 檢核點

- 圖上的數字與 freq.csv 一致嗎？抽三個詞核對。
- 「希望」排第一，代表什麼？還是只是口頭禪？
- 文字雲的字大小，能拿來精確比較兩個詞嗎？

單元 5｜情感分析：詞典法 vs. Gemini

目標：同一批評論用兩種方法判斷情感，與人工標註比較準確率，並分析誤判原因。

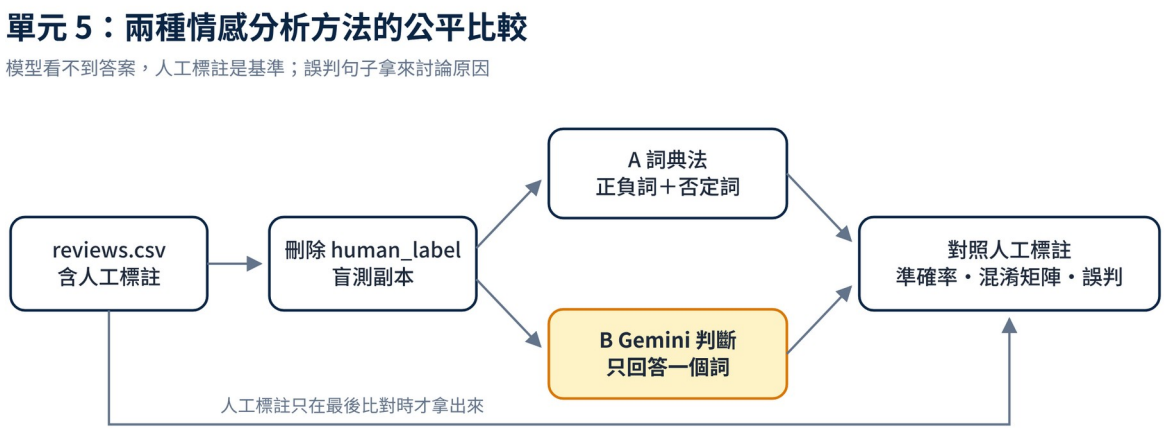


圖 8　兩種情感分析方法的公平比較

1. 兩人一組，先人工標註 reviews.csv 的「正面／負面／中性」，計算兩人一致率。
2. 請 Gemini 寫詞典法函式，要求先說明如何處理「不太好」「沒有難懂」這類否定句。
3. 執行方法 B：以 `google.colab.ai` 呼叫 Gemini 判斷，模型只看得到刪除標註後的盲測資料。

4. 比較兩方法準確率與混淆矩陣，挑 5 句誤判逐句寫原因。

Gemini 提示（CRISP）

C：reviews.csv 是課程評論，欄位「評論」與人工標註 human_label。
I：寫詞典法情感函式 lexicon_label(text)：正面詞加分、負面詞扣分，需處理否定詞「不、沒、未、別」。
S：回傳「正面／負面／中性」其中之一。
P：先說明你會怎麼處理「不太好」與「沒有想像中難懂」，等我同意再寫程式。

Colab 程式碼儲存格

```
# 單元 5-A：情感詞典法（含否定詞）
POS = ["好", "棒", "清楚", "實用", "喜歡", "有趣", "推薦", "有收穫"]
NEG = ["差", "難懂", "無聊", "太快", "失望", "混亂", "聽不懂"]
NEGATORS = ("不", "沒", "未", "別")

def lexicon_label(text):
    score = 0
    words = jieba.lcut(text)
    for i, w in enumerate(words):
        if any(p in w for p in POS):
            s = 1
        elif any(n in w for n in NEG):
            s = -1
        else:
            s = 0
        prev = words[i-1] if i > 0 else ""          # 前一個詞
        negated = w.startswith(NEGATORS) or prev.startswith(NEGATORS)
        if s and negated and not w.startswith("聽不"):
            s = -s                                  # 遇到否定詞就反轉
        score += s
    return "正面" if score > 0 else "負面" if score < 0 else "中性"
```

Colab 程式碼儲存格

```
# 單元 5-B：用 Gemini 判斷（Colab 內建，免金鑰）
from google.colab import ai
print(ai.list_models())                  # 先看看可用模型

def llm_label(text):
    prompt = ("判斷下列課程評論的情感，只回答「正面」「負面」「中性」其中一個詞。\n"
              "評論：" + text)
    ans = ai.generate_text(prompt).strip()
    return ans if ans in ("正面", "負面", "中性") else "無法判斷"

full = pd.read_csv("reviews.csv")        # 欄位：評論, human_label
blind = full.drop(columns=["human_label"])  # 盲測：模型看不到人工標註
blind["lex"] = blind["評論"].apply(lexicon_label)
blind["llm"] = blind["評論"].apply(llm_label)
blind["human"] = full["human_label"]
for m in ["lex", "llm"]:
    print(m, "準確率", round((blind[m] == blind["human"]).mean(), 2))
pd.crosstab(blind["human"], blind["llm"])
```

範例資料中，「沒有想像中難懂，蠻有趣的」被詞典法判為**中性**（人工標註為正面）。斷詞結果是「沒有／想像中／難懂／蠻／有趣」：程式只檢查前一個詞，「沒有」隔了「想像中」，否定沒有作用；「難懂」扣 1 分、「有趣」加 1 分，相互抵銷。這類句子正是課堂討論的好材料。

HITL 檢核點

- 人工標註兩人不一致的句子，能當標準答案嗎？
- 盲測資料真的刪掉 human_label 了嗎？打開 blind 確認欄位。
- 準確率高就能上線嗎？哪一種誤判代價最大？

預習影音：V11

單元 6｜把好流程存起來：規則、函式與範本筆記本

目標：把重複的規則寫進 Custom Instructions，把流程包成函式，存成範本筆記本，換資料只要一行。

1. 在 Gemini 面板更新 Custom Instructions：加入斷詞規則、輸出格式、不可修改測試等規則。
2. 在筆記本最上方建立文字儲存格「專案說明」：資料來源、欄位、規則與驗證方法（CRISP 的 C）。
3. 請 Gemini 把單元 2–4 的程式整理成 `analyze()` 函式，先看它的計畫。
4. 換一份新資料，一行呼叫完成分析。
5. 編輯 → 清除所有輸出，再 檔案 → 在雲端硬碟中儲存副本，作為範本筆記本。

Colab 程式碼儲存格

```
# 單元 6：把流程包成函式，換資料只要一行
def analyze(csv_path, col, top_n=15):
    df = pd.read_csv(csv_path)
    words = [w for t in df[col].dropna() for w in seg(t)
             if len(w) >= 2 and not w.isdigit()]
    top = pd.DataFrame(Counter(words).most_common(top_n), columns=["詞", "次數"])
    plt.figure(figsize=(8, 5))
    plt.barh(top["詞"][::-1], top["次數"][::-1], color="#0E7490")
    plt.title(f"{csv_path}｜{col}｜高頻詞")
    plt.tight_layout(); plt.show()
    return top

analyze("feedback.csv", "建議")
```

HITL 檢核點

- 函式有沒有寫死檔名或欄位？換資料試一次。
- 新資料套用舊的停用詞與自訂詞典合適嗎？
- 寫進 Custom Instructions 的規則，誰審核過？

單元 7｜LLM 當 NLP 引擎：批次摘要、分類與幻覺檢查

目標：用程式批次呼叫 Gemini，對每篇新聞產生摘要、主題與關鍵句，並用程式自動檢查幻覺。

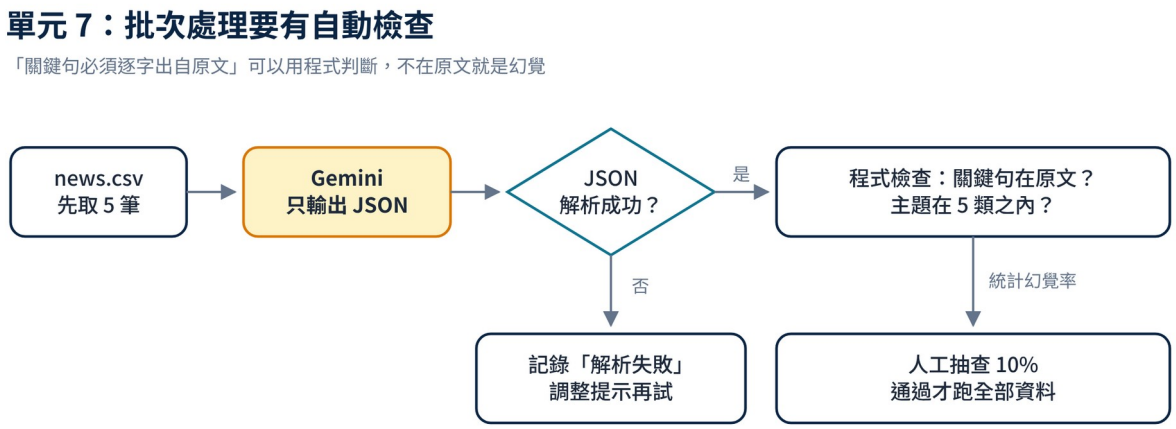


圖 9　批次處理與自動檢查流程

1. 準備 news.csv（欄位：內文），先只取前 5 筆。
2. 執行批次儲存格；Gemini 必須只輸出 JSON。
3. 程式自動檢查：關鍵句是否逐字出現在原文、主題是否在 5 類之內。
4. 人工抽查 10%，通過後才跑全部資料，並記錄呼叫次數。
5. （進階）需要更大用量時，改用步驟 6 的 API 金鑰路線。

Colab 程式碼儲存格

```
# 單元 7：批次摘要＋主題分類＋幻覺檢查
import json
TOPICS = ["病蟲害", "水產", "氣候", "市場", "其他"]

def tag(text):
    prompt = f"""閱讀下列農業新聞，只輸出 JSON，不要任何其他文字：
{{"summary": "50 字內摘要", "topic": "{'、'.join(TOPICS)} 擇一",
  "key_sentence": "逐字摘自原文的一句"}}
新聞：{text}"""
    raw = ai.generate_text(prompt).strip()
    raw = raw.removeprefix("```json").removesuffix("```").strip()
    try:
        return json.loads(raw)
    except json.JSONDecodeError:
        return {"summary": "", "topic": "解析失敗", "key_sentence": ""}

news = pd.read_csv("news.csv")            # 欄位：內文
rows = []
for text in news["內文"].head(5):         # 先跑 5 筆，人工確認後再全跑
    r = tag(text)
    ks = r.get("key_sentence", "")
    r["關鍵句在原文"] = bool(ks) and ks in text   # 不在原文＝幻覺
    r["主題合法"] = r.get("topic") in TOPICS
    rows.append(r)
pd.DataFrame(rows)
```

為什麼要檢查「關鍵句在原文」

LLM 會自信地編造。要求「逐字摘自原文」後，就能用一行程式 `ks in text` 判斷真假。

不在原文的比例，就是本次任務的一種幻覺率，請寫進監督日誌。

HITL 檢核點

- 關鍵句不在原文的比例是多少？
- 隨機抽 10 筆，主題分類與你的判斷一致幾筆？
- 遇到用量上限時，你會縮小批次、縮短提示，還是改用金鑰？

預習影音：V7、V11、V12（V12 使用舊版套件，程式請以本講義為準）

單元 8｜整合專題：文件問答小工具

目標：先用程式找出相關段落，再讓 Gemini 只根據段落回答並標出處；找不到就回答「資料中沒有」。

單元 8：文件問答（簡易檢索增強生成）

先由程式找證據，再讓 Gemini 根據證據回答；沒有證據就拒答

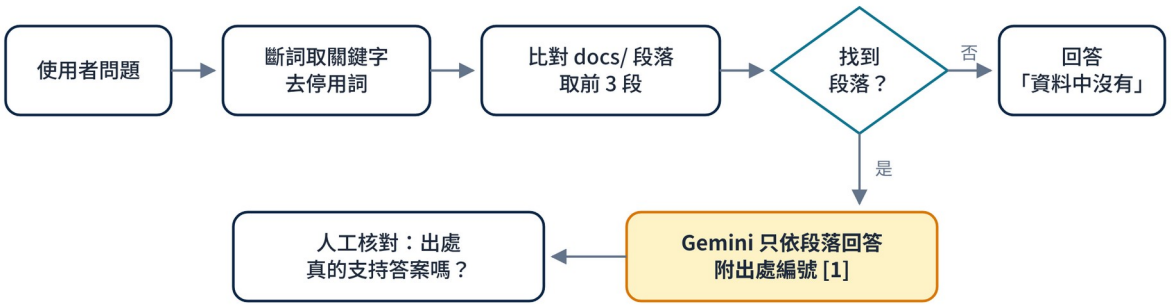


圖 10　文件問答：簡易檢索增強生成

1. 把課程公告或講義存成 docs 資料夾內的 .txt 檔，段落之間空一行。
2. 寫 10 題測試問題，其中 3 題是資料裡沒有的，預期答案由你填寫。
3. 執行問答儲存格，逐題檢查是否附出處、該拒答的是否拒答。
4. 請 Gemini 以「程式審查員」角色檢查程式，只列問題不改程式；你決定採納或退回。

Colab 程式碼儲存格

```
# 單元 8：文件問答（關鍵字檢索＋Gemini 依資料回答）
import glob, os
docs = []
for path in glob.glob("docs/*.txt"):
    for para in open(path, encoding="utf-8").read().split("\n\n"):
        if para.strip():
            docs.append({"source": os.path.basename(path), "text": para.strip()})

def retrieve(q, k=3):
    qw = {w for w in jieba.lcut(q) if len(w) >= 2 and w not in stop}
    scored = [(len(qw & set(jieba.lcut(d["text"]))), d) for d in docs]
    scored = sorted([x for x in scored if x[0] > 0], key=lambda x: -x[0])
    return [d for _, d in scored[:k]]

def ask(q):
    hits = retrieve(q)
    if not hits:
        return "資料中沒有", []
    context = "\n".join(f"[{i+1}] ({d['source']}) {d['text']}"
                        for i, d in enumerate(hits))
    prompt = f"""只根據下列資料回答，三句以內，句末標出處編號如 [1]。
資料不足就回答「資料中沒有」。
資料:
{context}
問題: {q}"""
    return ai.generate_text(prompt), [d["source"] for d in hits]

answer, sources = ask("期中考是哪一週? ")
print(answer, sources)
```

專題題目	資料來源	核心技術
課程 FAQ 問答	課程公告、講義	檢索＋Gemini 回答
病蟲害描述查詢	病蟲害說明文件	斷詞、自訂詞典、關鍵字檢索
水質報告摘要	監測報告	批次摘要＋警示規則
自選	經教師核可	—

HITL 檢核點

- 每個答案都附出處嗎？出處段落真的支持答案嗎？
- 問資料裡沒有的問題，工具會說「資料中沒有」，還是編答案？
- 審查員指出的問題，哪些採納、哪些退回？理由寫進監督日誌。

期末展示

每組 5 分鐘：現場執行筆記本 2 分鐘、展示一個「AI 犯錯被我們抓到」的例子 2 分鐘、監督日誌重點 1 分鐘。

5　評量設計

成績看「學生如何監督 AI」多於「程式寫得多好」。

項目	比重	內容	繳交方式
單元作品	20%	單元 1–7 各一份筆記本（含輸出）	雲端硬碟分享連結＋截圖上傳
期末筆記本	20%	可執行的文本分析或文件問答	清除個資後分享
提示日誌	20%	每單元至少 3 筆：提示原文、Gemini 回應摘要、我的修改	每週上傳
監督日誌	25%	HITL 檢核點回答：驗證方法、發現的錯誤、採納／修改／退回	每週上傳
展示與互評	15%	5 分鐘展示＋同儕互評	期末課堂

等級	驗證方法	錯誤發現	決策說明
優（90+）	自設測試資料並與人工結果比對	找出 AI 錯誤並說明原因與類型	每個決定都有依據
良（80–89）	有比對，但樣本少	找到錯誤，原因說明不完整	多數決定有依據
可（70–79）	只看程式有無執行成功	未發現或未記錄錯誤	決定缺少理由
待加強	未驗證	—	全盤接受 AI 結果

6 影音資源

用手機掃描 QR code 即可觀看。V1–V7 為本課程自製影片，V8–V12 為外部教學影片；外部影片的介面畫面可能與現行版本略有不同，操作以本講義為準。

#	QR	影片與連結	長度	對應	觀看重點
V1		**先思考後驗證 120 秒影音（英配雙語字幕）** https://youtu.be/yXTW5BtXPvQ	1:59	課程導論	課程核心方法「先思考、後驗證」
V2		**由作而管：人機協作三層級（150 秒精簡版）** https://youtu.be/jtSpZbgRm5g	2:25	課程導論	人機協作由協作到管理的三個層級
V3		**paul@CT&GEN AI COLAB** https://youtu.be/l5luM0T6CfM	0:58	課前準備	運算思維、生成式 AI 與 Colab 的實作概覽
V4		**心智圖 IPO 教學影片（120 秒）** https://youtu.be/RgvnGZE6dMY	2:01	單元 2	用心智圖拆解問題、寫出 IPO
V5		**由 IPO 架構到 Python 運作之 AI Video** https://youtu.be/_Pda5WXlGpI	0:56	單元 2	從 IPO 架構到 Python 程式
V6		**Smart Aquaculture Tutorial（180s）** https://youtu.be/O5Vdxg89dNE	3:01	單元 3 魚塭案例	智慧水產養殖情境，搭配魚塭案例

#	QR	影片與連結	長度	對應	觀看重點
V7		**基於 ROUGE 指標評估生成式 AI 模型之偏差與雜訊** https://youtu.be/glkNRqyLSio	0:56	單元 7	用 ROUGE 指標評估生成式 AI 的偏差與雜訊
V8		**Get started with Google Colaboratory（TensorFlow，英文）** https://youtu.be/inN8seMm7UI	3:10	課前準備	Colab 是什麼、如何建立與執行筆記本（官方）
V9		**Python 第一課：Google Colaboratory 介紹（DataSci Ocean）** https://youtu.be/eJCXFIoOwdw	23:40	課前準備	Colab 限制、開啟方式、介面與快捷鍵
V10		**How to Use Google Gemini in Google Colab for FREE（Valley4Techs，英文）** https://youtu.be/XWJzlzMaG1k	4:17	課前準備／單元 1	在 Colab 開啟 Gemini 協助寫程式
V11		**在 Google Colab 上設定 Gemini 與操作 Python（An-Wen Deng）** https://youtu.be/mUYWpvAncjk	16:53	單元 5、7	在 Colab 設定 Gemini 並以 Python 操作
V12		**5 分鐘在 Colab 快速使用 Gemini 來產生文本（李政軒）** https://youtu.be/zrKyTYQqH-s	8:19	單元 7（舊版套件）	以 API 金鑰生成文本；使用舊版套件，程式以本講義為準

課程頻道： youtube.com/@paulkuo4106　**短網址：** pjkuo.github.io/yt

課程頻道

7　常見問題

狀況	可能原因	處理方式
找不到 Gemini 圖示	帳號未滿 18 歲，或學校帳號未開放 AI 功能	改用個人帳號，或由教師示範；也可改走 API 金鑰路線
圖表中文變方框	未安裝或未指定中文字型	執行單元 4 的字型安裝儲存格

狀況	可能原因	處理方式
讀 CSV 出現亂碼	Excel 存成 Big5	`pd.read_csv("檔名", encoding="cp950")`，再另存 UTF-8
重新開啟後檔案不見	/content 為暫存空間	改放雲端硬碟並掛載（課前準備步驟 4）
變數或函式找不到	執行階段重啟後狀態清空	執行階段 → 全部執行，依序重跑
generate_text 出現用量錯誤	免費帳號每月用量上限	縮小批次、隔天再試，或改用 API 金鑰
Gemini 給的程式用 google.generativeai	舊版套件	要求改用 `google.colab.ai` 或 `google-genai`
測試通過但結果怪	測試太少或預期值被改	檢查 expected，預期值一律由人填寫

8 參考資料

Colab 與 Gemini 功能依 2026-11-01 查核之官方資料；版本更新快，以官方頁面為準。

- Google Colab 常見問題（AI 功能、年齡、資料使用、用量限制）：https://research.google.com/colaboratory/faq.html
- Customize your Gemini agent in Colab（Custom Instructions、Learn Mode，2026-04-08）：https://blog.google/innovation-and-ai/technology/developers-tools/colab-updates/
- All Colab users now get access to Gemini and Gemma models via Colab Python library（2025-11-12）：https://medium.com/google-colab/all-colab-users-now-get-access-to-gemini-and-gemma-models-via-colab-python-library-at-no-cost-a392599977c4
- Gemini API：模型清單：https://ai.google.dev/gemini-api/docs/models
- Gemini API：文字生成（google-genai）：https://ai.google.dev/gemini-api/docs/text-generation
- Google AI Studio：建立 API 金鑰：https://aistudio.google.com/apikey
- jieba 中文斷詞（含 dict.txt.big 說明）：https://github.com/fxsjy/jieba
- CKIP Transformers（中研院繁體中文斷詞）：https://github.com/ckiplab/ckip-transformers